

透過 Private Service Connect 端點，使用 Python SDK 在 Vertex AI 存取 Anthropic Claude Opus 4.6 和 Gemini 3.1

來源：<https://codelabs.developers.google.com/codelabs/anthropic-on-vertexai-psc?hl=zh-tw>

步驟數：10

透過 Python SDK 和 PSC 端點，從 VM 存取 Vertex AI 上的 Anthropic Opus 4.6 和 Gemini 3.1 Pro

目錄

1. [總覽](#)
2. [目標](#)
3. [設定環境](#)
4. [在 Vertex Model Garden 中啟用 Anthropic](#)
5. [建立 NAT 閘道和 VM](#)
6. [設定 VM 並進行測試](#)
7. [建立 googleapis 的 PSC 端點](#)
8. [手動建立 googleapis 的 DNS 項目](#)
9. [透過 IP 位址驗證端點連線](#)
10. [恭喜](#)

1. 總覽

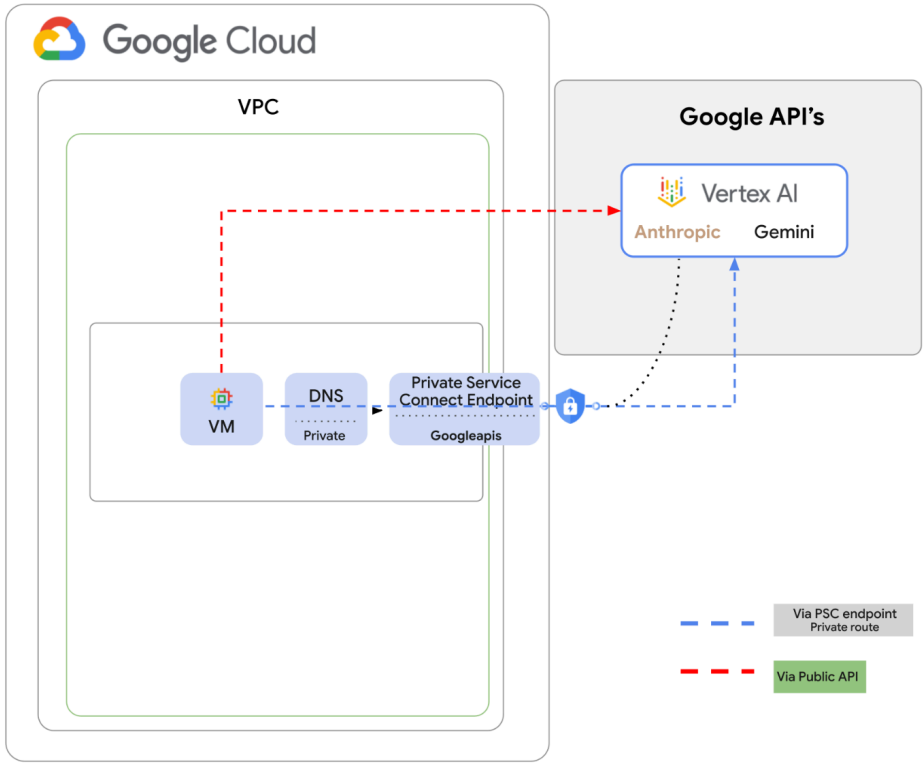
Vertex AI API 可透過網際網路存取，但您可能希望在企業中私下存取 Vertex AI API，而不透過網際網路。在本實驗室中，您將透過公開網際網路，在 VM 執行個體上執行 Python SDK，並透過 Vertex 存取 Anthropic Claude Opus 4.6 和 Gemini 3.1 Pro。

接著，您將建立 Googleapis 的 Private Service Connect 端點，並變更流量流程，使用私人端點連線至 Vertex API。

Python 程式碼範例會接收輸入問題，並傳送至 Claude Opus 生成答案，然後將輸出內容傳送至 Gemini 3 建立文字摘要並生成圖片。

在本實驗室中，您將建立下列模式。

圖 1。



2. 目標

在本實驗室中，您將瞭解如何執行下列工作：

- 設定 VM 執行個體以使用 Python SDK
- 在 Vertex AI 中啟用 Anthropic 模型資訊卡
- 透過 Python 指令碼連線至 **Anthropic Claude Opus 4.6** 和 **Gemini 3.1 Pro**
- 設定 PSC 端點以連線至 Googleapis
- 設定手動 DNS 項目
- 驗證 Googleapis 的連線路徑
- 執行 Python 指令碼來查詢模型

設定實驗室

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud console interface. At the top, there is a 'Select a project' dropdown menu. Below it, the 'Select a project' page is displayed, featuring a search bar labeled 'Search projects and folders'. To the right of the search bar is a 'NEW PROJECT' button. Below the search bar, the 'New Project' form is shown. It includes a 'Project name' field with the value 'My Project 13475', a 'Project ID' field with the value 'inbound-analogy-389614' (highlighted with a blue box), and a 'Location' field. The 'Project ID' field has a note: 'it cannot be changed later. EDIT'. At the bottom of the form are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

- 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：

A screenshot of a web browser window showing the Cloud Shell terminal. The browser tab is titled 'qwiklabs-gcp-b3de7b9575cb8d19'. The terminal has a black background with white text. It displays a welcome message and the user's prompt.

```
Welcome to Cloud Shell! Type "help" to get started.
google87764_student@qwiklabs-gcp-b3de7b9575cb8d19:~$
```

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

3. 設定環境

請注意，啟用部分資源會產生費用。請務必先瞭解這點再執行。為降低費用，請務必在完成實驗室後刪除資源。建議您先查看帳單，確認是否能接受這筆費用。💡

我們將建立含有防火牆規則的自訂虛擬私有雲。如果您已有 VPC 和專案，可以略過這個部分。

開啟控制台右上方的 Cloud Shell。然後依照下列方式設定：

Search (/) for resources, docs, products and more

Search



1. 啟用本實驗室中會用到的一些 API

```
gcloud services enable dns.googleapis.com
gcloud services enable aiplatform.googleapis.com
gcloud services enable servicedirectory.googleapis.com
```

2. 設定一些變數。這些變數是自訂 VPC 的專案 ID 和網路 ID (您會在步驟 4 中建立 VPC)。

```
projectid=$(gcloud config get-value project)
networkid=anthropic-net
echo $projectid
echo $networkid
```

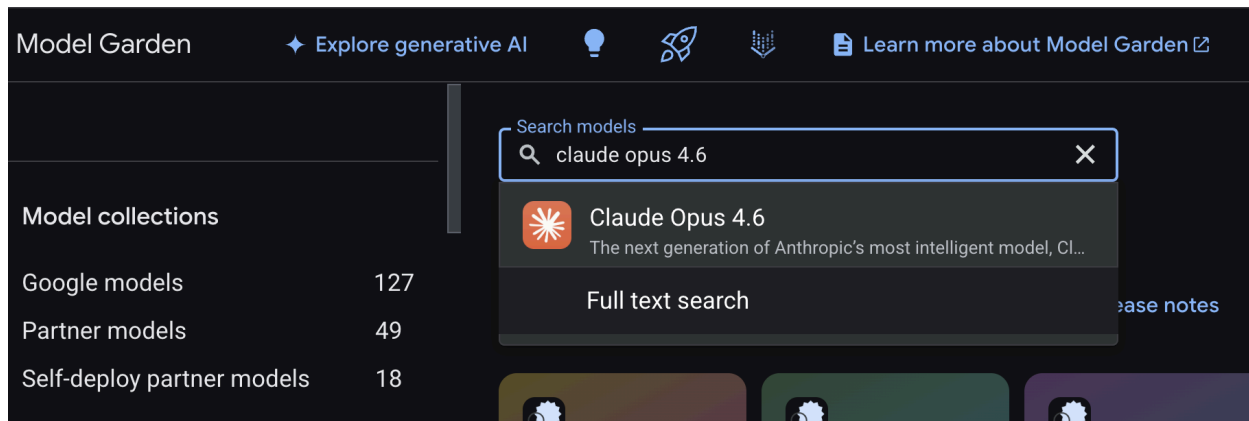
3. 現在建立自訂虛擬私有雲 (anthropic-net)、子網路 (vm1-subnet) 和防火牆規則

```
gcloud compute networks create $networkid \  
--project=$projectid \  
--subnet-mode=custom \  
--mtu=1460 \  
--bgp-routing-mode=global  
  
gcloud compute networks subnets create vm-subnet \  
--project=$projectid --range=10.0.88.0/24 \  
--stack-type=IPV4_ONLY --network=$networkid \  
--region=us-east1  
  
gcloud compute firewall-rules create $networkid-allow-icmp --project=$projectid \  
--network=$networkid \  
--description="Allows ICMP connections from any source to any instance on the network." \  
--direction=INGRESS \  
--priority=65534 \  
--source-ranges=0.0.0.0/0 \  
--action=ALLOW \  
--rules=icmp  
  
gcloud compute firewall-rules create $networkid-allow-ssh \  
--project=$projectid \  
--network=$networkid \  
--description="Allows TCP connections from any source to any instance on the network using  
port 22." \  
--direction=INGRESS --priority=65534 \  
--source-ranges=0.0.0.0/0 --action=ALLOW \  
--rules=tcp:22  
  
gcloud compute firewall-rules create $networkid-allow-web \  
--project=$projectid \  
--network=$networkid \  
--description="Allows TCP connections on port 8080." \  
--direction=INGRESS --priority=1000 \  
--source-ranges=0.0.0.0/0 --action=ALLOW \  
--rules=tcp:8080
```

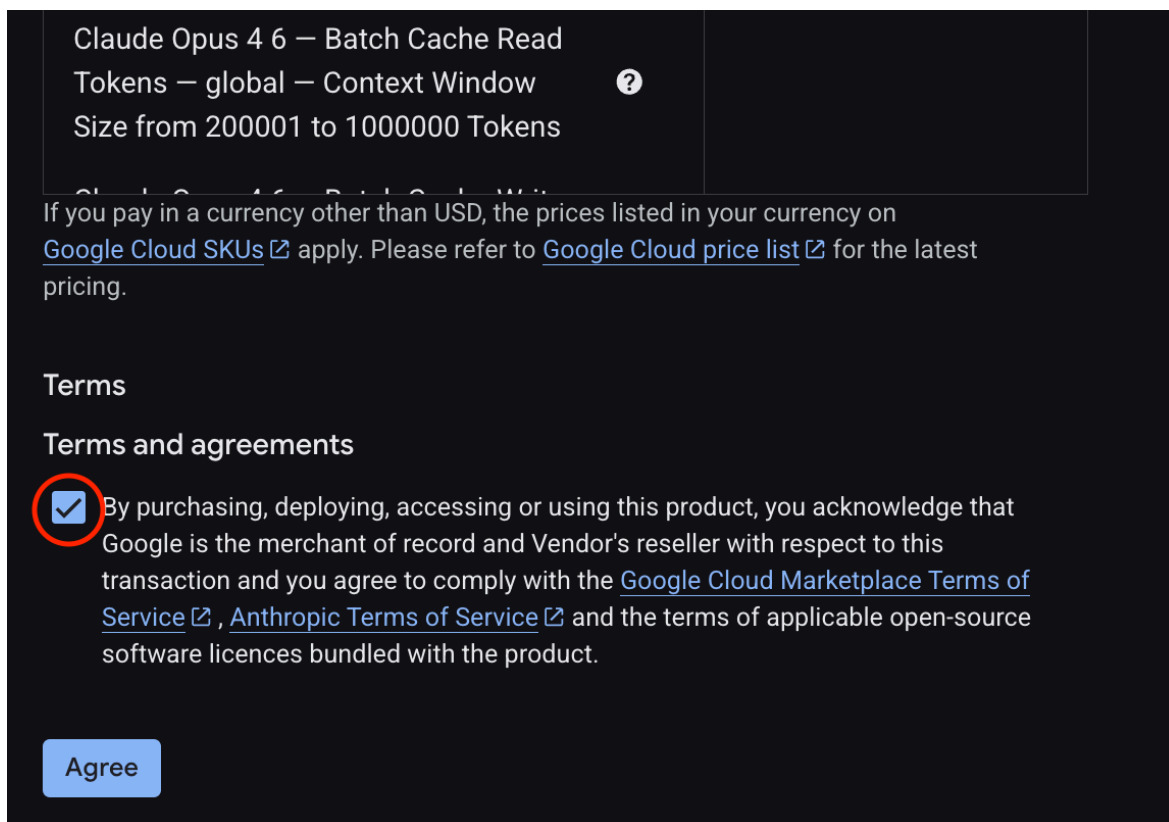
4. 在 Vertex Model Garden 中啟用 Anthropic

我們需要授予連出網際網路的外部存取權，因此請建立並附加 Cloud NAT 閘道。

1. 前往 **Vertex AI 資訊主頁**，然後選取「Model Garden」
2. 搜尋「Anthropic」，然後選取「Claude Opus 4.6」



3. 選取 **啟用**，系統會要求你填寫一些資訊。填寫表單並選取 **下一步**
4. 在最後一頁選取「同意」，即可啟用 Claude 4.6 Opus



5. 畫面上應會顯示成功頁面。

Successfully purchased Claude Opus 4.6



Now that you've agreed to the terms, you can enable Claude Opus 4.6 in Vertex AI

[Manage on Vertex AI](#)

注意：任何型號的裝置都適用相同程序

5. 建立 NAT 閘道和 VM

我們需要授予連出網際網路的外部存取權，因此請建立並附加 Cloud NAT 閘道。

在 **Cloud Shell** 中使用下列指令

1. 建立 Cloud NAT 和 Cloud NAT 閘道

```
gcloud compute routers create anthro-out-nat \  
  --network $networkid \  
  --region us-east1  
  
gcloud compute routers nats create anthro-out-nat-gw \  
  --router-region us-east1 \  
  --router anthro-out-nat \  
  --nat-all-subnet-ip-ranges \  
  --auto-allocate-nat-external-ips
```

現在讓我們建立一個 VM，透過 **Python SDK** 存取 **Vertex AI** 上的 **Anthropic**。

2. 在同一個 Cloud Shell 工作階段中，使用下列指令建立 **anthro-vm**。(請等待 3 到 4 分鐘，讓 VM 啟動並安裝開機指令碼)

```
gcloud compute instances create anthro-vm \  
  --project=$projectid \  
  --zone=us-east1-b \  
  --network-interface=stack-type=IPv4_ONLY,subnet=vm-subnet,no-address,network=$networkid \  
  --metadata startup-script="#! /bin/bash  
  apt-get update  
  apt-get install python3 python3-dev python3-venv tcpdump dnsutils -y  
  python3 -m venv /opt/py-anthro-env  
  /opt/py-anthro-env/bin/pip install -U google-genai 'anthropic[vertex]' rich  
  chmod -R 777 /opt/py-anthro-env"
```

您應該會看到建立的 VM 沒有公開 IP 位址。現在來設定 VM 的

6. 設定 VM 並進行測試

1. 在 Google Cloud Shell 中，透過 SSH 連線至名為 **anthro-vm** 的新 VM。(您可能需要等待 3 分鐘，確保 VM 完全設定完成)

```
gcloud compute ssh anthro-vm \
  --project=$projectid \
  --zone=us-east1-b \
  --tunnel-through-iap \
  -- -L 8080:localhost:8080
```

2. 連線後，請快速測試是否能連線至 Vertex Gemini API。

```
dig *-aiplatform.googleapis.com
```

3. 畫面應如下所示 (地址會有所不同)。請注意，由於 API 是公用 API，因此路徑會經過公開 IP 位址。

```
; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> *-aiplatform.googleapis.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 3728
;; flags: qr rd ra; QUERY: 1, ANSWER: 16, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 512
;; QUESTION SECTION:
*-aiplatform.googleapis.com.    IN      A

;; ANSWER SECTION:
*-aiplatform.googleapis.com. 300 IN    A      172.217.204.95
*-aiplatform.googleapis.com. 300 IN    A      172.217.203.95
*-aiplatform.googleapis.com. 300 IN    A      173.194.215.95
*-aiplatform.googleapis.com. 300 IN    A      142.250.98.95
*-aiplatform.googleapis.com. 300 IN    A      173.194.217.95
*-aiplatform.googleapis.com. 300 IN    A      142.251.107.95
*-aiplatform.googleapis.com. 300 IN    A      74.125.196.95
```

4. 現在請啟動 **venv** 環境：

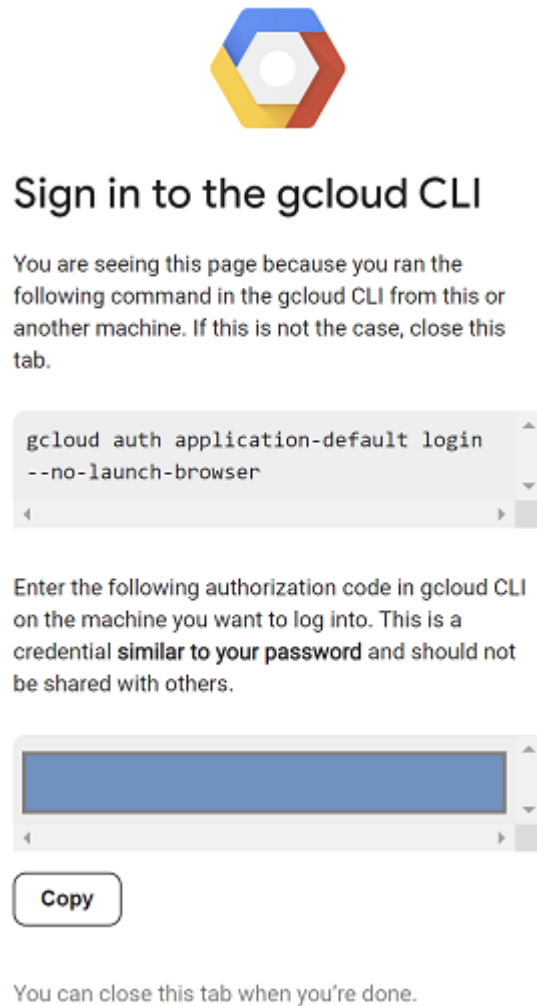
```
source /opt/py-anthro-env/bin/activate
```

5. 現在請驗證這項設定，以便稍後進行測試。在 VM 中執行下列指令，並在系統提示時按下 **y**。

```
gcloud auth application-default login
```

6. 接著複製開頭為 **https://** 的網址，在實驗室瀏覽器視窗中開啟新分頁並貼上網址。接受提示。

7. 看到下列選取複製畫面時，請切換回 VM 「anthro-vm」 工作階段，然後在「Enter authorization code:」中貼上複製的授權碼，並按下 Enter 鍵進行驗證。



8. 現在，我們要在 `env` 中使用 Python。這會建立 Python 檔案 `super-gen-ai.py`，向 **Claude Opus 4.6** 詢問「**建構代理式 AI 的主要步驟有哪些？**」然後將輸出內容傳遞給 **Gemini 3.1 Pro**，建立摘要，接著 **Gemini 3 Pro Image** 會生成圖片，並儲存在 VM 上。

複製並貼上下列內容。

[illegible]

```

"""
console.print(super_me_art)
console.rule("[bold gold1]★ LEVEL 1-1: Vertex AI Orchestrator ★[/bold gold1]")

def main():
    os.system('clear') # Clears the terminal for a clean UI
    print_super_me_header()

    # Create the images directory
    os.makedirs("images", exist_ok=True)

    with console.status("[bold yellow]🌐 Collecting Coins (Authenticating)...[/bold
yellow]", spinner="super_me"):
        try:
            creds, project_id = google.auth.default()
            LOCATION = "global"
        except Exception as e:
            console.print(f"[bold red]💥 GAME OVER! Authentication failed:[/bold red] {e}")
            return

    with console.status(f"[bold green]🦖 Powering up Clients for {project_id}...[/bold
green]", spinner="super_me"):
        google_client = genai.Client(vertexai=True, location=LOCATION, project=project_id)
        claude_client = AnthropicVertex(region=LOCATION, project_id=project_id)

    console.print("\n[bold cyan]The Kingdom needs a strategy![/bold cyan]")

    # Auto-run prompt without user interaction
    question = "What are the key steps to building Agentic AI?"

    console.print(Panel(question, title="[bold green]Green Dino (Claude Opus) is entering the
pipe...[/bold green]", border_style="green"))

    with console.status("[bold green]🟢 Green Dino is clearing the level...\n[/bold green]",
spinner="super_me"):
        try:
            msg = claude_client.messages.create(
                model="claude-opus-4-6",
                max_tokens=2048,
                messages=[{"role": "user", "content": question}]
            )
            claude_text = msg.content[0].text
            console.print(f"[bold green]✓ Level Cleared! Green Dino generated
{len(claude_text)} bytes of strategy.[/bold green]\n")
        except Exception as e:
            console.print(f"[bold red]💥 BOSS ATTACK (Claude API Error):[/bold red] {e}")
            return

    with console.status("[bold red]🔴 Red Hero (Gemini 3.1 Pro Preview) is summarizing the
map...\n[/bold red]", spinner="super_me"):
        text_prompt = f"""
        I am providing you with a response generated by Claude.

```

```

1. READ the text below.
2. GENERATE a concise, bulleted executive summary.

--- INPUT TEXT ---
{claude_text}
"""

try:
    text_response = google_client.models.generate_content(
        model="gemini-3.1-pro-preview",
        contents=text_prompt,
        config=types.GenerateContentConfig(temperature=0.7)
    )
    console.print("[bold red]✓ Super Me! Text Summary Complete.[/bold red]")
except Exception as e:
    console.print(f"[bold red]🔥 CHOMPER PLANT (Gemini Text Error):[/bold red] {e}")
    return

with console.status("[bold blue]🌀 Mushroom Friend (Gemini 3 Pro Image) is painting the
castle...\n[/bold blue]", spinner="super_me"):
    image_prompt = f"""
    CREATE a high-quality, futuristic illustration of autonomous AI agents for a tech
    blog header (16:9 aspect ratio).
    Base the image directly on the following text summary:

    {text_response.text}
    """

    try:
        image_response = google_client.models.generate_content(
            model="gemini-3-pro-image-preview",
            contents=image_prompt,
            config=types.GenerateContentConfig(
                response_modalities=['TEXT', 'IMAGE'],
                temperature=0.7
            )
        )
        console.print("[bold blue]✓ Masterpiece Complete![/bold blue]\n")
    except Exception as e:
        console.print(f"[bold red]🔥 FALLING BLOCK (Gemini Image Error):[/bold red]
{e}")
        return

console.rule("[bold gold1]▶ COURSE CLEAR! ▶[/bold gold1]")

if text_response.text:
    console.print(Panel(text_response.text.strip(),
                        title="[bold gold1]Gemini 3.1 Pro Summary[/bold gold1]",
                        border_style="gold1"))

for part in image_response.parts:
    if image := part.as_image():

```

```

        # Save inside the images folder with the focus-specific name
        filename = "images/building-agentic.png"
        image.save(filename)
        console.print(f"\n[bold green]✓ REWARD SAVED:[/bold green] [underline]
{os.path.abspath(filename)}[/underline]")

    console.rule("[bold red]Web Server Warp Pipe[/bold red]")
    with console.status("[bold yellow]Opening the warp pipe (HTTP server)...[/bold yellow]",
spinner="super_me"):
        # Kill any existing server first so it doesn't crash on port binding
        os.system('pkill -f "http.server" > /dev/null 2>&1')

        # Start the server with the root directory set specifically to 'images'
        subprocess.Popen(
            ["python3", "-m", "http.server", "8080", "--directory", "images"],
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL
        )

        console.print("[bold green]✓ Warp Pipe is open in the background![/bold green]")
        console.print("[bold magenta]View your files here (Click Web Preview -> Preview on port
8080):[/bold magenta] [underline]http://localhost:8080[/underline]\n")

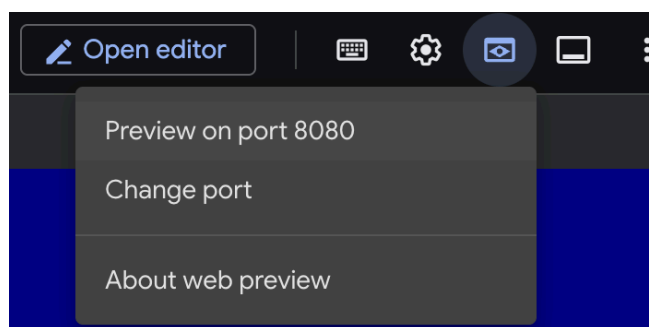
if __name__ == "__main__":
    main()
EOF

python3 super-gen-ai.py

```

指令碼會在有趣的介面中執行，並啟動網路伺服器，方便您查看圖片輸出內容。如要再次執行指令碼，請輸入 `python3 super-gen-ai.py`。

9. 在 Cloud Shell 中選取「網頁預覽」選項，然後選取「透過以下通訊埠預覽：8080」



10. 在開啟的網頁工作階段中，選取「building_agentic.png」 **building_agentic.png**即可查看生成的圖片。(以下為 AI 生成的圖片範例)



11. 在 **Cloud Shell** 中執行下列指令，即可退出網路伺服器。

```
kill -f "http.server"
```

12. 接著輸入 `exit`，返回 Cloud Shell 首頁。完成後，請繼續下一個步驟。

注意：你可以將提示問題修改為其他內容。

7. 建立 googleapis 的 PSC 端點

如要啟用與 Vertex API 端點的私人連線，請為 googleapis 建立 Private Service Connect 端點。這樣一來，我們就能使用指派的私人 IP 位址，將流量轉送至所需的 googleapis (在本例中為 Vertex Gemini)。

1. 如果尚未開啟 Cloud Shell，請開啟。我們會為 PSC 端點建立 IP，並建立 Private Service Connect 端點。在本例中，我們將使用 **192.168.255.230**。

```
projectid=$(gcloud config get-value project)
networkid=anthropic-net

gcloud compute addresses create anthro-ip \
  --global \
  --purpose=PRIVATE_SERVICE_CONNECT \
  --addresses=192.168.255.230 \
  --network=$networkid

gcloud compute forwarding-rules create pscanthrovertex \
  --global \
  --network=$networkid \
  --address=anthro-ip \
  --target-google-apis-bundle=all-apis
```

2. 驗證

```
gcloud compute addresses list --filter="name=( 'anthro-ip' ...)"

gcloud compute forwarding-rules describe pscanthrovertex --global
```

8. 手動建立 googleapis 的 DNS 項目

您可以使用私人 DNS 建立手動 DNS 項目，指向 PSC 端點。這會影響您指派給該廣告活動的所有聯播網。您現在要建立下列項目

- 為 googleapis.com 建立**私人 DNS 區域**，並附加至 anthropic-net 網路。
- A 記錄，將根網域對應至 Private Service Connect 端點 IP (192.168.255.230)
- 萬用字元 CNAME 記錄，可將所有子網域 (例如 aiplatform.googleapis.com) 重新導向至根 A 記錄

1. 在 Cloud Shell 中執行下列指令。

```
gcloud dns managed-zones create googleapis-private \
  --description="Private DNS zone for googleapis.com" \
  --dns-name="googleapis.com." \
  --visibility="private" \
  --networks="anthropic-net"

gcloud dns record-sets create "googleapis.com." \
  --zone="googleapis-private" \
  --type="A" \
  --ttl="300" \
  --rrdatas="192.168.255.230"

gcloud dns record-sets create "*.googleapis.com." \
  --zone="googleapis-private" \
  --type="CNAME" \
  --ttl="300" \
  --rrdatas="googleapis.com."
```

2. 列出區域中的記錄集，確認 A 記錄和 CNAME 已正確建立。

```
gcloud dns record-sets list --zone="googleapis-private"
```

3. 在控制台檢視畫面中，應如下所示。您應該會看到設定，其中包含 **A** 記錄和 **CNAME**，如下所示：

Cloud DNS

Cloud CDN

Cloud NAT

Cloud Service Mesh (Traff...

Service Directory

Cloud Domains

Private Service Connect

SSL policies

Service extensions

googleapis-private

DNS name

googleapis.com.

Description

Private DNS zone for googleapis.com

Type

Private

Record sets

In use by

+ Add standard

+ Add with routing policy

Delete record sets

Refresh

Filter

Filter record sets

☐	DNS name ↑	Type	TTL (seconds)	Record data
☐	*.googleapis.com.	CNAME	300	googleapis.com.
☐	googleapis.com.	SOA	21600	ns-gcp-private.googledomains.com. 1
☐	googleapis.com.	NS	21600	ns-gcp-private.googledomains.com.
☐	googleapis.com.	A	300	192.168.255.230

4. 接著，我們會在 **anthro-vm** 上驗證這些變更的連線能力

9. 透過 IP 位址驗證端點連線

讓我們使用私人端點連線至 **Gemini**。

1. 在 Google Cloud Shell 中，透過 **SSH** 連線至名為 **anthro-vm** 的新 VM。

```
gcloud compute ssh anthro-vm \  
  --project=$projectid \  
  --zone=us-east1-b \  
  --tunnel-through-iap \  
  -- -L 8080:localhost:8080
```

2. 在 VM 中，使用 **dig** 指令檢查 **aiplatform-pscanthrovertex.p.googleapis.com** 的連線路徑。您應該會看到 PSC 端點的 IP **192.168.255.230**

```
dig aiplatform-pscanthrovertex.p.googleapis.com +noall +answer
```

讓我們使用私人端點連線至 **Gemini**。

3. 現在讓我們執行 **dig**，看看是否可以連線至 Vertex Gemini API。

```
dig *-aiplatform.googleapis.com
```

4. 畫面應會顯示類似內容 (地址會有所不同)。請注意，路徑是透過 PSC 端點 IP 位址。

```
; <<>> DiG 9.18.41-1-deb12u1-Debian <<>> *-aiplatform.googleapis.com  
;; global options: +cmd  
;; Got answer:  
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 33703  
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1  
  
;; OPT PSEUDOSECTION:  
; EDNS: version: 0, flags;; udp: 512  
;; QUESTION SECTION:  
*-aiplatform.googleapis.com. IN A  
  
;; ANSWER SECTION:  
*-aiplatform.googleapis.com. 300 IN CNAME googleapis.com.  
googleapis.com. 300 IN A 192.168.255.230  
  
;; Query time: 8 msec  
;; SERVER: 169.254.169.254#53(169.254.169.254) (UDP)  
;; WHEN: Sat Nov 29 15:19:15 UTC 2025  
;; MSG SIZE rcvd: 86
```

5. 使用 **ping** 指令。我們可以快速測試一下。注意：這個 IP 是 PSC 端點，因此 Ping 會失敗。

```
ping -c 2 aiplatform.googleapis.com
```

6. 現在請啟動 venv 環境：

```
source /opt/py-anthro-env/bin/activate
```

7. 現在，我們要在 `env` 中使用 Python。這會建立 Python 檔案 `star-me.py`，向 Claude Opus 4.6 詢問「**Give me a step by step recipe to make a roti for a beginner?**」(請提供適合初學者的印度烤餅食譜，並逐步說明做法)。然後將輸出內容傳送至 **Gemini 3.1 Pro**，生成摘要，並由 **Gemini 3 Pro Image** 生成圖片，儲存至 VM


```

os.makedirs("images", exist_ok=True)

with console.status("[bold yellow]🌐 Bypassing Security (Authenticating)...[/bold
yellow]", spinner="star_me"):
    try:
        creds, project_id = google.auth.default()
        LOCATION = "global"
    except Exception as e:
        console.print(f"[bold red]💥 COMM LINK SEVERED! Authentication failed:[/bold
red] {e}")
        return

    with console.status(f"[bold cyan]🔒 Establishing secure connection for {project_id}...
[/bold cyan]", spinner="star_me"):
        google_client = genai.Client(vertexai=True, location=LOCATION, project=project_id)
        claude_client = AnthropicVertex(region=LOCATION, project_id=project_id)

    console.print("\n[bold cyan]The Galaxy needs sustenance![/bold cyan]")

    # Auto-run prompt for the recipe
    question = "Give me a step by step recipe to make a roti for a beginner?"

    console.print(Panel(question, title="[bold green]Space Master (Claude Opus) is searching
the archives...[/bold green]", border_style="green"))

    with console.status("[bold green]🟢 Space Master is compiling the ancient
texts...\n[/bold green]", spinner="star_me"):
        try:
            msg = claude_client.messages.create(
                model="claude-opus-4-6",
                max_tokens=2048,
                messages=[{"role": "user", "content": question}]
            )
            claude_text = msg.content[0].text
            console.print(f"[bold green]✓ Archives Retrieved! Space Master generated a
{len(claude_text)}-character Holocron.[/bold green]\n")
        except Exception as e:
            console.print(f"[bold red]💥 INTERFERENCE DETECTED (Claude API Error):[/bold
red] {e}")
            return

    with console.status("[bold gold]🟡 Protocol Bot (Gemini 3.1 Pro Preview) is translating
the recipe...\n[/bold gold]", spinner="star_me"):
        text_prompt = f"""
I am providing you with a response generated by a Space Master.
1. READ the text below.
2. GENERATE a concise, bulleted executive summary.

--- INPUT TEXT ---
{claude_text}
"""

```



```

try:
    text_response = google_client.models.generate_content(
        model="gemini-3.1-pro-preview",
        contents=text_prompt,
        config=types.GenerateContentConfig(temperature=0.7)
    )
    console.print("[bold gold1]✓ Translation Complete! The Protocol Bot has
summarized the steps.[/bold gold1]")
except Exception as e:
    console.print(f"[bold red]✖ SYSTEM FAILURE (Gemini Text Error):[/bold red]
{e}")
    return

with console.status("[bold blue]🌀 Astro Bot (Gemini 3 Pro Image) is projecting the
hologram...\n[/bold blue]", spinner="star_me"):
    image_prompt = f"""
    CREATE a high-quality, beautiful illustration for a blog header (16:9 aspect ratio)
of someone making roti, but with a subtle sci-fi, cinematic lighting aesthetic.
Base the image directly on the following text summary:

{text_response.text}
"""

try:
    image_response = google_client.models.generate_content(
        model="gemini-3-pro-image-preview",
        contents=image_prompt,
        config=types.GenerateContentConfig(
            response_modalities=['TEXT', 'IMAGE'],
            temperature=0.7
        )
    )
    console.print("[bold blue]✓ Hologram Projection Ready![/bold blue]\n")
except Exception as e:
    console.print(f"[bold red]✖ BAD COMPRESSOR (Gemini Image Error):[/bold red]
{e}")
    return

console.rule("[bold cyan]▶ MISSION ACCOMPLISHED ▶[/bold cyan]")

if text_response.text:
    console.print(Panel(text_response.text.strip(),
                        title="[bold gold1]Protocol Bot Recipe Summary[/bold gold1]",
                        border_style="gold1"))

for part in image_response.parts:
    if image := part.as_image():
        # Save inside the images folder with the focus-specific name
        filename = "images/cookingroti.png"
        image.save(filename)
        console.print(f"\n[bold green]✓ DATA SECURED:[/bold green] [underline]
{os.path.abspath(filename)}[/underline]")

```

```

console.rule("[bold cyan]Hyperlink Uplink[/bold cyan]")
with console.status("[bold yellow]Opening the frequencies (HTTP server)...[/bold yellow]", spinner="star_me"):
    # Kill any existing server first so it doesn't crash on port binding
    os.system('pkill -f "http.server" > /dev/null 2>&1')

    # Start the server with the root directory set specifically to 'images'
    subprocess.Popen(
        ["python3", "-m", "http.server", "8080", "--directory", "images"],
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL
    )

    console.print("[bold green]✓ Transmitting on Port 8080![/bold green]")
    console.print("[bold magenta]View your files here (Click Web Preview -> Preview on port 8080):[/bold magenta] [underline]http://localhost:8080[/underline]\n")

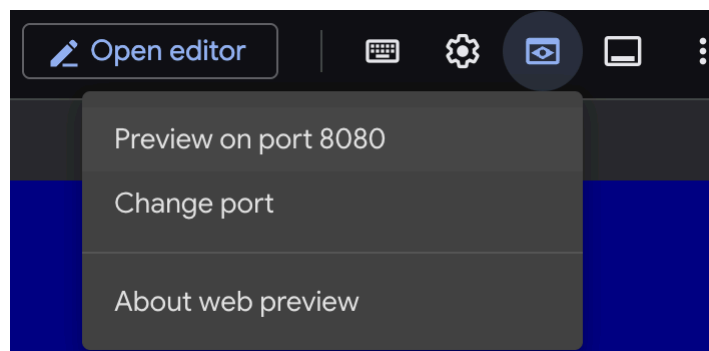
if __name__ == "__main__":
    main()
EOF

python3 star-me.py

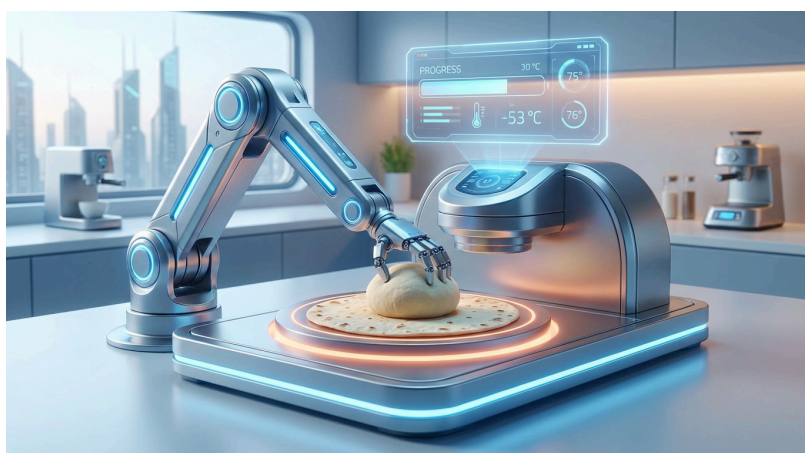
```

指令碼會在有趣的介面中執行，並啟動網路伺服器，方便您查看圖片輸出內容。如要再次執行指令碼，請輸入 `python3 star-me.py`

8. 現在前往 Cloud Shell 中的網頁預覽選項，選取「透過以下通訊埠預覽：8080」



9. 選取「cookingroti.png」 **cookingroti.png**，即可查看生成的圖片。(以下為 AI 生成的範例)



10. 在 **Cloud Shell** 中執行下列指令，即可退出網路伺服器並返回 VM。

```
kill -f "http.server"
```

11. 接著輸入 `exit`，返回 Cloud Shell 首頁。

10. 恭喜

恭喜！您已成功連線至 Vertex 上的 **Anthropic Claude Opus 4.6** 和 **Gemini 3.1 Pro**，並透過公用 API 位址和 Private Service Connect Endpoint for Googleapis 私下連線。這項功能可將非公開 API 連線能力擴展至地端部署/其他雲端環境，這些環境透過 (互連網路、Cross-Cloud Interconnect 和虛擬私有雲) 連線。

清除所用資源

```
gcloud compute instances delete anthro-vm --zone=us-east1-b --quiet

gcloud compute routers nats delete anthro-out-nat-gw --router=anthro-out-nat --region=us-east1 --quiet

gcloud compute routers delete anthro-out-nat --region=us-east1 --quiet

gcloud compute firewall-rules delete anthropic-net-allow-icmp anthropic-net-allow-ssh anthropic-net-allow-web --quiet

gcloud compute forwarding-rules delete pscanthrovertex --global --quiet

gcloud dns record-sets delete googleapis.com. --zone googleapis-private --type A --quiet

gcloud dns record-sets delete *.googleapis.com --zone googleapis-private --type CNAME --quiet

gcloud dns managed-zones delete googleapis-private --quiet

gcloud compute addresses delete anthro-ip --global --quiet

gcloud compute networks subnets delete vm-subnet --region=us-east1 --quiet

gcloud compute networks delete anthropic-net --quiet
```

後續步驟/瞭解詳情

如要進一步瞭解 Vertex AI 網路，請參閱[這篇文章](#)。

Codelab：透過 Private Service Connect 端點，使用 Python SDK 存取 Gemini 3 Pro 對話

Codelab：使用 ADK 建構 AI 代理：基礎知識

挑戰下一個實驗室

繼續完成 Google Cloud 任務，或查看其他 Google Cloud Skills Boost 實驗室：

- [Google Cloud 網路基礎知識](#)
- [探索生成式 AI - Vertex AI](#)
- [在 Google Cloud 實作代理式 AI](#)